LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 7: Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Jawab:

Isolasi Kontainer (*Isolated Namespace*): Setiap kontainer Docker berjalan di dalam lingkungan atau jaringan virtual yang terisolasi sepenuhnya satu sama lain. Masing-masing kontainer memiliki *loopback interface* (localhost atau 127.0.0.1) sendiri-sendiri.

Pencarian Alamat yang Keliru: Ketika kontainer node-red dikonfigurasi untuk terhubung ke localhost, sistem akan menerjemahkannya sebagai "cari layanan di dalam kontainer Node-RED itu sendiri", bukan di kontainer mqtt-broker.

Terjadinya *Connection Refused*: Karena layanan broker MQTT (Mosquitto) tidak aktif atau tidak berjalan di dalam kontainer Node-RED, maka permintaan koneksi tersebut ditolak oleh sistem, sehingga menghasilkan error *Connection Refused*.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

-

Jawab :

NodeMCU berada di jaringan Wi-Fi dengan alamat IP 192.168.43.102 (subnet 192.168.43.x), sedangkan laptop tempat broker Mosquitto berjalan memiliki alamat IP 192.168.1.15 (subnet 192.168.1.x).

Perangkat yang berada pada *subnet* atau *broadcast domain* yang berbeda tidak dapat saling berkomunikasi secara langsung pada lapisan jaringan lokal (Layer 2/3) tanpa adanya perangkat *router* atau aturan *routing* khusus. Akibatnya, paket data MQTT dari NodeMCU tidak akan pernah sampai ke tujuan IP laptop.

Menghubungkan NodeMCU dan laptop ke **jaringan Wi-Fi atau titik akses (*Access Point*) yang sama persis**—misalnya, keduanya sama-sama dikonfigurasi untuk terhubung ke satu sumber *hotspot* seluler yang sama atau router Wi-Fi lokal yang sama.

Memastikan bahwa pada pengaturan *router* atau *hotspot* yang digunakan, fitur **AP Isolation** (atau *Client Isolation*) dalam keadaan **nonaktif** agar komunikasi antar-perangkat tidak saling diblokir.

atihan 3 (Implementasi Filter Histeresis pada Rule Engine):
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

Jawab :

Pengiriman Telemetri (Sensor ke Broker):

- Sensor DHT11 pada NodeMCU membaca suhu dan kelembapan setiap 4 detik.
- Data dikemas dalam format JSON ({"suhu": xx, ...}) dan dikirim via MQTT ke topik kampus/labiot/telemetri melalui broker Mosquitto di Docker (192.168.1.15).

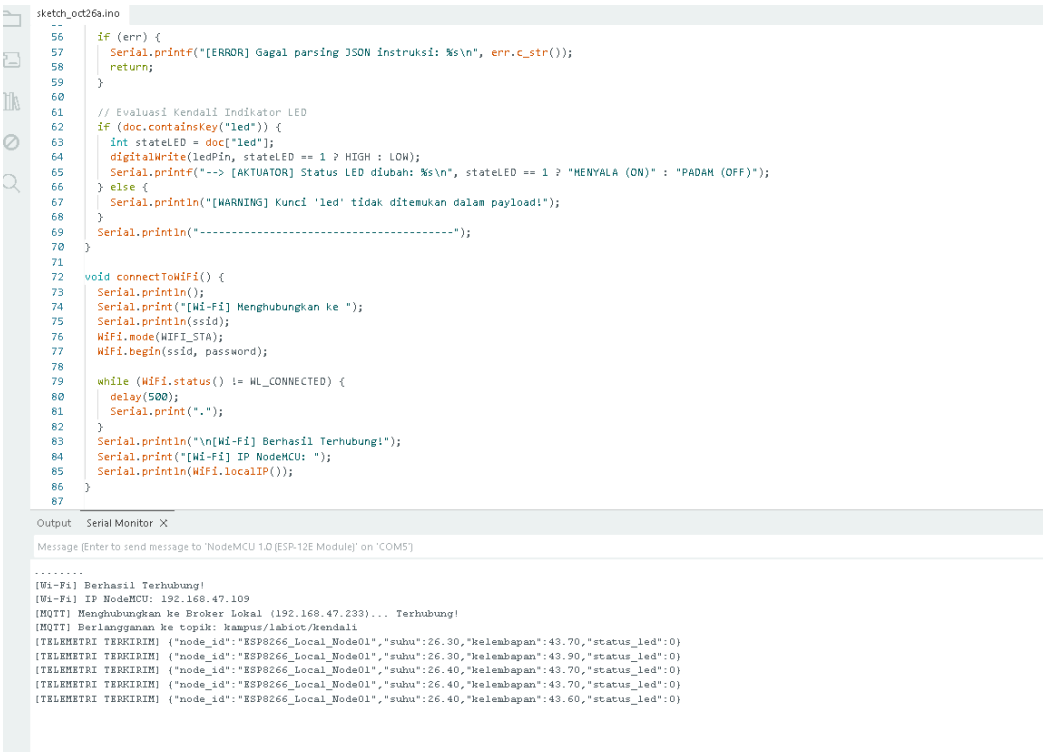
Pemrosesan & Filter Histeresis (Node-RED):

- Node-RED menerima data suhu dan memprosesnya menggunakan aturan histeresis di *Function Node* untuk mencegah kedipan cepat (*flickering*):
  - **LED Menyala (1)**: Jika suhu > 31.5°C.
  - **LED Padam (0)**: Jika suhu < 30.0°C.

- **Status Dipertahankan**: Jika suhu berada di rentang 30.0°C – 31.5°C, status LED sebelumnya tetap dikunci menggunakan memori flow (flow.get / flow.set).

Eksekusi Kendali (Broker ke Aktuator):

- Keputusan status LED dari Node-RED dipublikasikan ke topik kampus/labiot/kendali.
- NodeMCU yang *subscribe* ke topik tersebut menerima perintah JSON, lalu fungsi *callback* mengeksekusinya secara fisik untuk mengubah pin LED (HIGH / LOW).



- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Jawab :

Seluruh alur kanvas visual (*flows*), kredensial, dan konfigurasi yang telah kamu buat di dalam Node-RED akan **hilang total (terhapus)**. Kontainer akan kembali ke kondisi awal (*fresh state*) karena tidak ada lagi direktori fisik di komputer host (./nodered:/data) yang menyimpan data tersebut secara persisten di luar siklus hidup kontainer.

Bind Mount: Mengaitkan direktori atau file dari *path* sistem operasi *host* komputer secara langsung ke dalam kontainer (contohnya ./nodered:/data). Pengguna memiliki akses penuh untuk membaca dan memodifikasi file tersebut secara langsung dari luar Docker.

Named Volume: Area penyimpanan khusus yang dikelola sepenuhnya oleh Docker di dalam ruang penyimpanan internal sistem Docker. Pengguna tidak mengakses file mentahnya secara langsung di folder biasa pada komputer *host*, dan metode ini umumnya lebih optimal untuk portabilitas serta performa lintas sistem operasi.